OSSP HACKATHON 2026 — FORMAL PROJECT REPORT

TLB and Page Table Architecture Simulator

Virtual-to-Physical Address Translation Engine & Memory Wall Performance Analysis

ABSTRACT: *Virtual memory is a cornerstone of contemporary operating systems, facilitating process protection, isolated addressing spaces, and transparent secondary storage paging. However, hierarchical in-memory page tables introduce a substantial latency bottleneck known as the Memory Wall. The Translation Lookaside Buffer (TLB) acts as a high-speed hardware cache to mitigate this translation penalty. This report documents the architecture, formal mathematical model, algorithm design, and empirical results of an interactive TLB and Page Table Simulator developed for the OSSP Hackathon.*

1. Problem Statement & Motivation

Modern computer architectures decouple application memory from physical DRAM via virtual memory. Every virtual memory reference must be resolved to a physical memory address before execution can proceed. Without hardware caching, this causes the classical **double-access penalty**: one memory cycle is consumed querying the page table in RAM, and a second cycle retrieves the actual data. In multi-level paging schemes (such as 4-level x86-64), this overhead scales up to 500%.

Objective: Develop a high-fidelity, interactive simulator that accurately models address translation, visualizes TLB hits and misses, implements cache replacement algorithms, handles page faults via demand paging, and quantifies Effective Access Time (EAT) in real time.

2. System Architecture & Mathematical Model

The system simulates a 16-bit address space (65,536 bytes) with configurable page sizes (256B, 1KB, or 4KB). Given page size P = 2^k bytes, the address decomposition follows:

```
Virtual Address [16-bit] = [ VPN (Bits 15..k) | Offset d (Bits k-1..0) ]
Physical Address = [ PFN (Frame #) | Offset d (Bits k-1..0) ]
Bit Calculations: Offset Bits k = log2(PageSize), VPN = VA >> k, Offset = VA & ((1 << k) - 1)
```

2.1 Effective Access Time (EAT) Formula

Effective Access Time measures the average memory access latency. With TLB Hit Ratio H, TLB access time t_TLB (2 ns), and RAM access time t_MEM (100 ns):

```
EAT = H × (t_TLB + t_MEM) + (1 - H) × (t_TLB + 2 × t_MEM) + (P_fault × t_fault)
• Without TLB (Baseline): 2 × 100 ns = 200.0 ns
• With 90% TLB Hit Rate: 0.90 × 102 ns + 0.10 × 202 ns = 112.0 ns (44% Latency Reduction)
```

3. Memory Management Algorithms & Replacement Policies

When the TLB cache is saturated (default 4 slots), a replacement policy dictates which entry is evicted. We implemented and benchmarked two policies:

Replacement Policy	Eviction Metric	Algorithmic Advantage	Known Tradeoff
Least Recently Used (LRU)	Monotonic access timestamp	Optimal for temporal locality and looping traces	Requires timestamp counter overhead
First-In, First-Out (FIFO)	Insertion sequence order	Zero hardware timestamp overhead; simple ring queue	Susceptible to Belady's Anomaly

4. Experimental Benchmark Results

Four synthetic workload benchmarks were executed to evaluate the cache behavior under varying access patterns:

Benchmark Suite	Access Pattern	TLB Hit Ratio	Observed EAT	Speedup
High Temporal Locality	Repetitive access to Pages 1 & 2	87.5%	114.5 ns	1.75x
Sequential Scan	Contiguous array memory pass	75.0%	127.0 ns	1.57x
Loop Exceeding TLB	5-page cyclic access (4 slots)	0.0% (Thrashing)	202.0 ns	0.99x
Uniform Random	Random 16-page lookups	12.5%	189.5 ns	1.05x

5. Team Contributions & Division of Roles

Team Member	Role & Core Responsibilities
Member 1 (Lead)	System Architecture, Bitwise Decomposition, Mathematical Model & EAT Formulation
Member 2	TLB Cache State Machine, Associative Lookup Engine, LRU & FIFO Algorithms
Member 3	Interactive Web Visualizer, Pipeline State Animation, Responsive CSS Design System
Member 4	Python CLI Simulator, Benchmark Workload Validation, Comprehensive Documentation

6. Conclusion

The TLB and Page Table Simulator successfully satisfies all Hackathon deliverables: a working dual-mode prototype, complete GitHub-ready codebase, mathematical validation, and benchmarked execution. By clearly demonstrating the 44% latency reduction achieved by hardware caching, this system provides a robust learning and experimentation tool for systems programming.